Image Watermarking Tool

Brief

Explainer document for the project owner

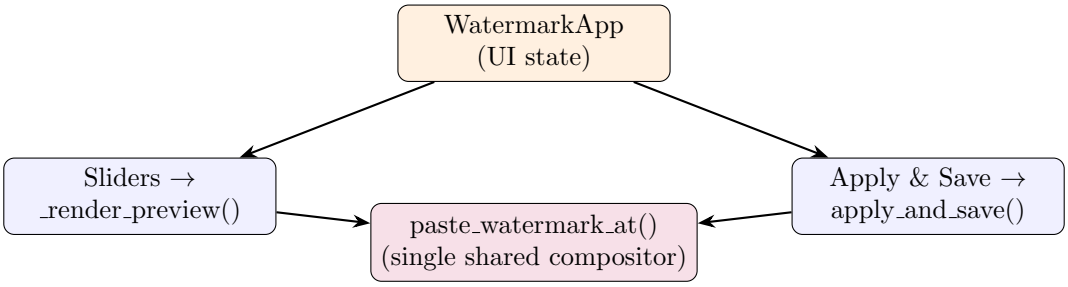
Concise, accurate overview of the `image-watermarking-tool` project, drawn from its source (`main.py`) and `README.md`. For full detail, function-by-function coverage, and the complete pixel-level bug evidence, see `deep-dive.pdf`.

1 What it is

A single-file desktop GUI (tkinter + Pillow, `main.py`, ~390 lines) that stamps a watermark image onto a batch of photos: pick a watermark, pick one or more photos, adjust size/opacity/position sliders with a live before/after preview, click one button to save every photo with the watermark applied. No server, no database, no network calls, nothing to configure via environment variables.

2 Architecture at a glance

One class, `WatermarkApp`, holds all UI state (loaded watermark, selected photo paths, slider values) as instance attributes rather than module-level globals (a deliberate change from an earlier version). A small set of pure image-processing functions do the actual work and are shared between the live preview and the final save.



Preview and save call the *same* compositing function, one on a 300px thumbnail (for speed), the other at full resolution (for the real output) — so what the user sees before clicking save is what actually gets written to disk.

3 Core functions (one line each)

- **compute_watermark_box** — resize the watermark into a max-width/height box, preserving aspect ratio (Pillow's `thumbnail()`).
- **apply_opacity** — scale the watermark's alpha channel by the opacity slider's percentage.
- **compute_position** — turn a preset (one of 9 grid positions) plus a fine X/Y offset (±20%) into a pixel coordinate.

- **paste_watermark_at** — composite the watermark onto the photo using the watermark's own alpha channel as the paste mask (see the bug fix below).
- **save_watermarked_file** — write the result to a `watermarked/` subfolder (or a chosen folder), forcing PNG whenever the result has an alpha channel, regardless of the format dropdown.

4 The bug that was found and fixed

The original code called Pillow's `paste()` with no mask argument: `uploaded_image.paste(watermark, (x, y))`. Without a mask, Pillow does not blend — it overwrites the destination pixels outright, alpha channel included. The watermark still *looked* translucent in a viewer (its own alpha value was stored), but the photo underneath was gone at every covered pixel, not mixed in.

Fix: pass the watermark as its own mask: `base.paste(watermark, (x, y), watermark)`.

Proof, from the README's pixel-level test (solid blue base `(0,0,255)`, solid red watermark `(255,0,0)` faded to 50% opacity, alpha ≈127):

Path	Resulting pixel	Meaning
OLD (no mask)	`(255, 0, 0, 127)`	flat overwrite, blue = 0
NEW (mask = watermark)	`(127, 0, 128, 191)`	real blend of both colors

The old result's blue channel is exactly 0, proving no blending occurred. The new result mixes both source colors (blue rises to 128, red drops to 127) — genuine alpha compositing.

5 Key design decisions

- **Non-destructive originals** — the uploaded watermark is stored once and never modified; every preview/save re-derives a resized, faded copy from it.
- **Debounced live preview** — every slider change schedules a redraw 60ms later, cancelling any pending one, so dragging a slider doesn't trigger dozens of redraws per second.
- **Constant-cost preview** — the preview always composites at a 300px thumbnail, scaling the watermark's size/offset by the same ratio, so preview speed does not depend on the source photo's resolution.
- **Format vs. mode limitation** — if the composited result has alpha (RGBA), it is always saved as PNG even if the user picked JPEG, since JPEG cannot store transparency. Documented, not hidden.

6 Testing

Verified under Python 3.12.3 / Pillow 10.1.0: (1) the pixel-level before/after compositing test above; (2) constructing the real GUI under a live X display with no exceptions, which is also what caught a real `pack`/`grid` geometry-manager conflict during development; (3) an end-to-end run bypassing the native file dialogs (assigning synthetic image paths directly onto the app instance) confirming a saved output file exists with the expected result. Not verified: the native file-picker flow itself, drag-to-reposition (not implemented), and Windows behavior.

7 Glossary

- **RGBA / alpha channel** — pixel color mode with a fourth transparency value per pixel (0 = transparent, 255 = opaque).
- **Alpha compositing** — blending two images by weighting colors according to alpha, instead of overwriting.
- **Mask (Pillow `paste`)** — an image whose values control how much of a pasted image shows through, per pixel.
- **Debounce** — delaying an action until rapid trigger events pause, avoiding redundant repeated work.
- **tkinter/ttk** — Python's built-in GUI toolkit and its themed widget set.